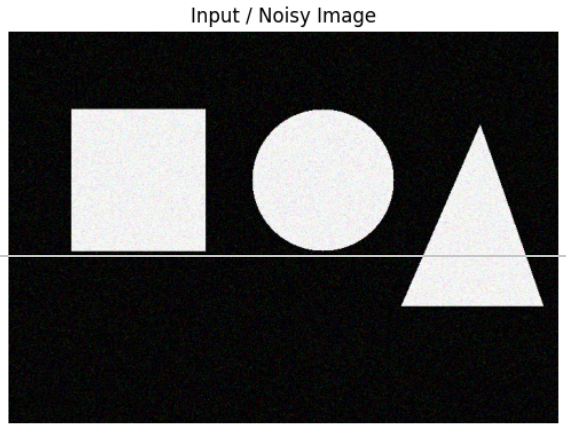
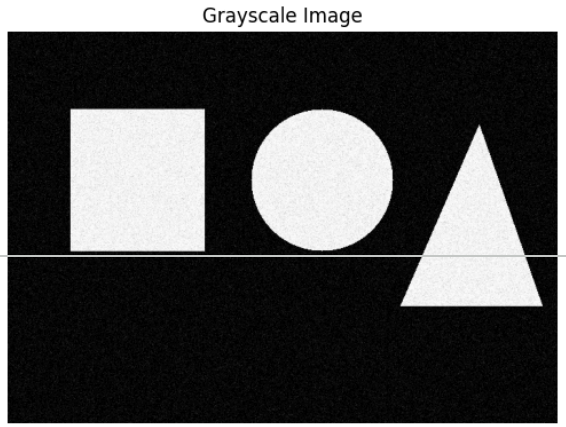
```
import cv2
import numpy as np
import matplotlib.pyplot as plt

# ========================================
# MULTI-STAGE IMAGE PROCESSING PIPELINE
# Preprocessing -> Edge Detection -> Feature Extraction
# ========================================

# Create a sample image
img = np.zeros((500, 700, 3), dtype=np.uint8)

# Draw objects
cv2.rectangle(img, (80, 100), (250, 280), (255, 255, 255), -1)
cv2.circle(img, (400, 190), 90, (255, 255, 255), -1)

pts = np.array([[500, 350], [600, 120], [680, 350]], np.int32)
cv2.fillPoly(img, [pts], (255, 255, 255))

# Add noise
noise = np.random.normal(0, 25, img.shape).astype(np.int16)
noisy_img = np.clip(img.astype(np.int16) + noise, 0, 255).astype(np.uint8)

# ========================================
# STAGE 1: PREPROCESSING
# ========================================

gray = cv2.cvtColor(noisy_img, cv2.COLOR_BGR2GRAY)

# Gaussian filtering to reduce noise
blur = cv2.GaussianBlur(gray, (5, 5), 0)

# ========================================
# STAGE 2: EDGE DETECTION
# ========================================

edges = cv2.Canny(blur, 50, 150)
```

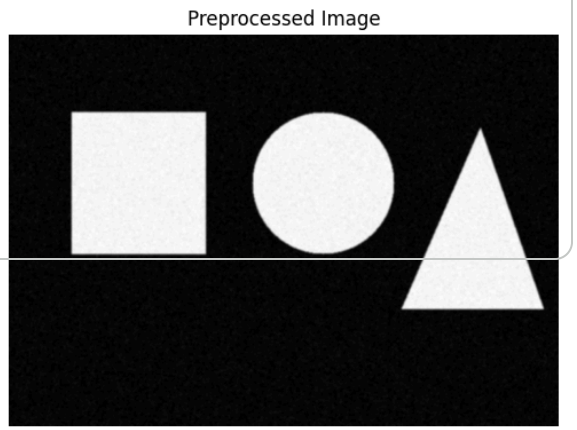
```
# ========================================
# STAGE 3: FEATURE EXTRACTION
# Using Shi-Tomasi corner detection
# ========================================

corners = cv2.goodFeaturesToTrack(
    blur,
    maxCorners=100,
    qualityLevel=0.01,
    minDistance=10
)

feature_image = cv2.cvtColor(edges, cv2.COLOR_GRAY2BGR)

corner_count = 0

if corners is not None:
    corners = np.int32(corners)

    for corner in corners:
        x, y = corner.ravel()

        cv2.circle(
            feature_image,
            (x, y),
            6,
            (0, 0, 255),
            -1
        )

        corner_count += 1

# ========================================
# DISPLAY ALL STAGES
# ========================================

plt.figure(figsize=(15, 10))
```

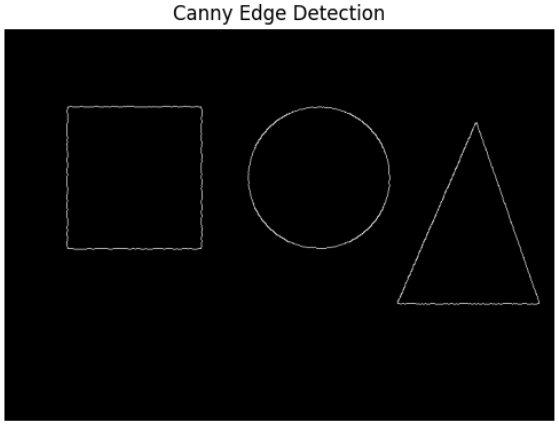
```
plt.subplot(2, 3, 1)
plt.imshow(cv2.cvtColor(noisy_img, cv2.COLOR_BGR2RGB))
plt.title("Input / Noisy Image")
plt.axis("off")

plt.subplot(2, 3, 2)
plt.imshow(gray, cmap="gray")
plt.title("Grayscale Image")
plt.axis("off")

plt.subplot(2, 3, 3)
plt.imshow(blur, cmap="gray")
plt.title("Preprocessed Image")
plt.axis("off")

plt.subplot(2, 3, 4)
plt.imshow(edges, cmap="gray")
plt.title("Canny Edge Detection")
plt.axis("off")

plt.subplot(2, 3, 5)
plt.imshow(cv2.cvtColor(feature_image, cv2.COLOR_BGR2RGB))
plt.title("Feature Extraction")
plt.axis("off")

# Pipeline diagram
plt.subplot(2, 3, 6)
plt.text(
    0.5, 0.80,
    "IMAGE PROCESSING PIPELINE",
    ha="center",
    fontsize=14,
    fontweight="bold"
)

plt.text(
    0.5, 0.60,
    "Input Image\n↓\nPreprocessing\n↓\nEdge Detection\n↓\nFeature Extraction",
    ha="center",
```

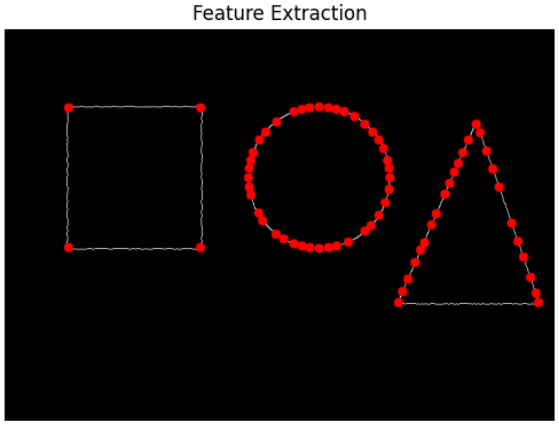
```
    va="center",
    fontsize=13
)

plt.axis("off")

plt.tight_layout()
plt.show()

# ========================================
# OUTPUT / ANALYSIS
# ========================================

print("=" * 55)
print("MULTI-STAGE IMAGE PROCESSING PIPELINE")
print("=" * 55)

print("\nStage 1 - Preprocessing")
print("Gaussian filtering reduces image noise.")

print("\nStage 2 - Edge Detection")
print("Canny detects important object boundaries.")

print("\nStage 3 - Feature Extraction")
print("Shi-Tomasi detects important corner points.")

print("\nRESULTS")
print("-" * 55)
print("Image Size              :", gray.shape)
print("Edge Pixels Detected    :", np.sum(edges > 0))
print("Features/Corners Found  :", corner_count)

print("\nCONCLUSION")
print("-" * 55)
print("Preprocessing improves image quality.")
print("Edge detection extracts object boundaries.")
print("Feature extraction identifies important points.")
print("Together, the stages improve the overall image")
print("processing and feature analysis performance.")
```


Input / Noisy Image


Grayscale Image


Preprocessed Image


Canny Edge Detection


Feature Extraction

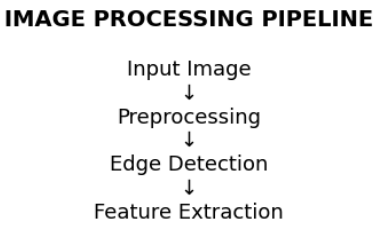


IMAGE PROCESSING PIPELINE
Input Image
↓
Preprocessing
↓
Edge Detection
↓
Feature Extraction

==========